# 武汉大学计算机学院
# 本科生课程设计报告

## HealthMate 系统设计说明

专业: 计算机弘毅班

课程: 计算机综合项目实践

指导教师: 谭小琼

团队名称: token

团队成员: 黎宇恒、周炜

学年学期: 2025-2026 学年第二学期

武汉大学计算机学院
2025 年 3 月 24 日

# 目录

1 引言 ..... 3
1.1 编写目的 ..... 3
1.2 系统定位 ..... 3
1.3 设计目标 ..... 3
2 系统体系架构 ..... 4
2.1 总体架构设计 ..... 4
2.2 分层详细说明 ..... 4
2.3 体系架构核心特点 ..... 5
3 系统功能结构 ..... 6
3.1 功能层次结构 ..... 6
3.2 功能模块详细说明 ..... 6
4 系统核心用例与时序设计 ..... 9
4.1 核心用例选择与定义 ..... 9
4.2 核心用例时序图 ..... 9
4.3 时序图业务逻辑说明 ..... 9
5 复杂功能算法设计 ..... 11
5.1 自然语言健康数据解析与双轨存储算法 ..... 11
5.2 AI 建议生成与任务难度动态调整算法 ..... 13
6 面向对象类设计 ..... 17
6.1 类设计核心思路 ..... 17
6.2 核心类图设计 ..... 17
6.3 主要类职责详细说明 ..... 17
7 接口设计 ..... 19
7.1 接口设计原则 ..... 19
7.2 统一响应格式 ..... 19
7.3 核心接口清单 ..... 19
7.4 接口安全设计 ..... 23
8 数据库物理设计 ..... 25
8.1 数据库选型及理由 ..... 25
8.2 数据库表结构设计 ..... 25
8.3 主要索引设计 ..... 29
8.4 表间关系设计 ..... 30
8.5 物理建表 SQL 示例 ..... 31
9 UI 界面与交互设计 ..... 33
9.1 UI 设计核心原则 ..... 33
9.2 视觉规范 ..... 33
9.3 核心页面结构设计 ..... 33
9.4 关键交互设计 ..... 37

# 1 引言

## 1.1 编写目的

本文档基于相关需求规格说明书与项目选题报告，详细说明 HealthMate 健康伴侣系统的技术设计细节，明确开发标准与实现规范，为编码、测试、部署及答辩提供统一技术依据，保障团队开发一致性与高效性。

## 1.2 系统定位

面向亚健康人群、健身小白及职场上班族的非医疗级健康管理系统，聚焦“轻量化管理”与“个性化干预”，以大语言模型为核心，实现“数据记录 – AI 分析 – 任务干预 – 打卡反馈 – 趋势可视化”闭环，不提供医疗级诊断与建议，仅作为日常健康习惯培养辅助工具，满足课程工程化开发要求。

## 1.3 设计目标

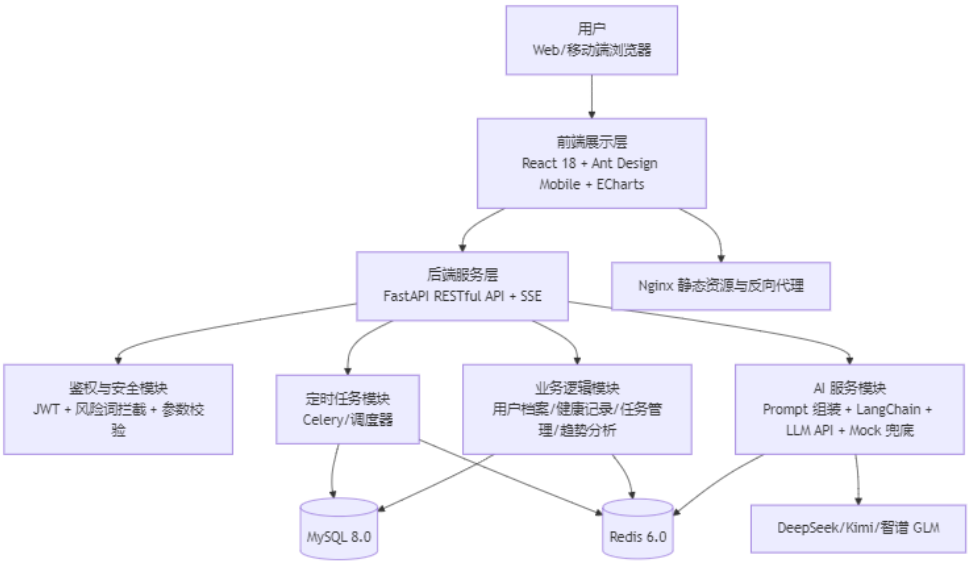
1. 降低录入门槛：支持自然语言与手动表单双模式录入，模糊描述可解析；
2. 个性化建议生成：融合 LLM、RAG 与长期记忆，生成贴合用户的连续建议；
3. 健康管理闭环：AI 建议自动转化为打卡任务，实现全流程管理；
4. 数据可视化：多维度图表展示趋势，自动标注异常点；
5. 安全与容错：建立风险词拦截、Prompt 约束、Mock 兜底等机制；
6. 工程化落地：遵循前后端分离、容器化部署等标准，确保可开发、可测试、可部署。

# 2 系统体系架构

## 2.1 总体架构设计

采用前后端分离架构，按展示层、业务层、AI 服务层、数据存储层、基础设施层分层设计，各层通过标准化接口交互，兼顾落地性与扩展性。

系统整体架构流程图如下：




## 2.2 分层详细说明

### 2.2.1 前端展示层

核心技术栈：React 18 + Ant Design Mobile + ECharts 5+ + Axios + SSE；

核心职责：页面展示、表单校验、用户交互、可视化渲染、流式建议展示、响应式适配；

设计规范：组件化开发，仅通过 API 与后端交互。

### 2.2.2 后端服务层

核心技术栈：Python 3.10+ + FastAPI + Celery + SQLAlchemy；

核心职责：请求路由、参数校验、JWT 鉴权、业务编排、事务控制、异常处理、

SSE 推送；

核心模块：鉴权安全、业务逻辑、定时任务。

### 2.2.3 AI 服务层

核心职责：模糊输入解析、Prompt 组装、个性化建议生成、任务结构化拆解、Mock 兜底、长期记忆管理（周总结压缩），与业务层解耦，支持大模型替换。

### 2.2.4 数据存储层

1. MySQL 8.0：存储用户档案、健康记录等结构化、强一致性数据，支持事务与 JSON 字段；
2. Redis 6.0：缓存预生成建议、周总结、JWT Token 等高频数据，提升响应速度，缓存命中率≥90%。

### 2.2.5 基础设施层

3. Docker + Docker Compose：容器化编排，支持一键启停；
4. Nginx：静态资源托管与反向代理；
5. 云服务器：阿里云/腾讯云学生服务器（Ubuntu 20.04 LTS，1 核 2G+）。

## 2.3 体系架构核心特点

1. 前后端分离，职责清晰，支持并行开发；
2. AI 服务封装化，可灵活切换大模型；
3. 双数据库协同，平衡性能与一致性；
4. 定时任务独立，支撑长期连续性；
5. 多层安全防护，控制 AI 医疗风险；
6. 容器化部署，可移植性强。

# 3 系统功能结构

## 3.1 功能层次结构

按用户操作流程与后台能力划分为 6 大核心模块，核心子功能必须实现，优化子功能后续落地：

## 3.2 功能模块详细说明

### 3.2.1 账号与档案管理模块

#### 核心子功能

1. 注册/登录：账号密码校验，JWT Token 生成（有效期 2 小时）；
2. 基础健康档案填写：新用户强制填写，含性别、身高、体重等核心信息；
3. 健康目标修改：支持目标切换，同步更新目标版本。

#### 优化子功能

1. 个人信息修改：实时同步数据库；
2. 密码修改：验证原密码，强制刷新 Token。

### 3.2.2 健康数据录入模块

#### 核心子功能

1. 自然语言录入：模糊描述解析为结构化数据；
2. AI 解析结果修正：用户可手动调整解析数据；
3. 风险词拦截：阻断高危医疗词汇输入；
4. 原始输入保留：原样存储用户自然语言输入。

#### 优化子功能

1. 手动表单录入：精确数据填写；
2. 语音录入：语音转文本后解析。

### 3.2.3 AI 智能分析与建议模块

核心功能（均需实现）：

1. 今日建议生成：融合多源上下文，避免模板化；
2. SSE 流式输出：打字机效果，支持断点续传；
3. 任务结构化拆解：生成 1-3 个轻量化打卡任务；
4. 长期记忆管理：周总结压缩（200 字内）；
5. 断签回归模式：连续未记录时生成恢复性建议；
6. 任务难度动态调整：基于昨日完成率适配难度与数量；
7. Mock 兜底：大模型不可用时返回预置建议；
8. 目标变更适配：忽略与新目标冲突的历史数据。

### 3.2.4 任务目标与打卡模块

#### 核心子功能

1. 今日任务展示：卡片式呈现任务与 AI 缘由；
2. 任务打卡/取消：事务控制，实时更新状态；
3. 完成率统计：日/周统计，支撑 AI 建议调整；
4. 历史打卡查询：按日期筛选；
5. 任务过期管控：超期不可打卡。

#### 优化子功能

1. 任务提醒：自定义提醒时间；
2. 成就系统：连续打卡标识。

### 3.2.5 健康数据可视化模块

#### 核心子功能

1. 健康仪表盘：展示核心数据与趋势图；
2. 周/月趋势分析：支持历史均值对比；
3. 健康标签统计：饼图/词云展示；
4. 异常点标注：红色标记异常数据；
5. 交互查询：图表悬浮查看详情，支持缩放。

### 优化子功能

1. 数据导出：Excel 格式导出；
2. 仪表盘定制：自主选择展示模块。

## 3.2.6 系统后台定时任务模块

核心功能（均需实现）：

1. 周记忆压缩：每周日凌晨生成周总结；
2. 次日建议预生成：每日凌晨 4 点缓存建议；
3. 打卡状态归档：每日凌晨 1 点归档前一日任务。

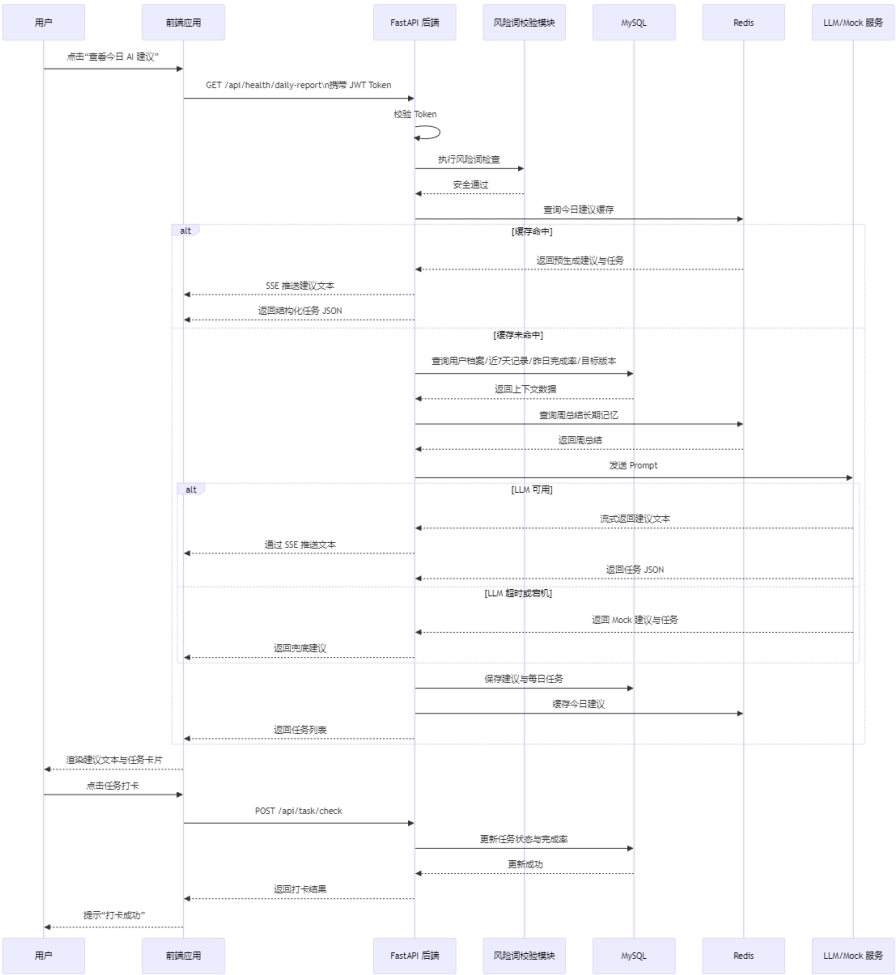
# 4 系统核心用例与时序设计

## 4.1 核心用例选择与定义

选择「获取 AI 今日健康建议并生成打卡任务」为核心用例，覆盖全业务链路与核心能力。

- 前置条件：用户登录鉴权、填写健康档案、系统服务正常；
- 后置条件：流式展示建议、任务存库、支持打卡操作。

## 4.2 核心用例时序图




## 4.3 时序图业务逻辑说明

1. 鉴权与安全校验前置，双重校验通过后进入流程；
2. 缓存优先，提升响应速度，未命中则触发完整流程；
3. 多源上下文组装 Prompt，确保建议个性化；
4. SSE 流式输出，缓解等待焦虑；
5. Mock 兜底保障可用性，超时 5 秒自动触发；
6. 事务控制确保数据一致性；
7. 打卡反馈数据支撑后续建议优化，形成闭环。

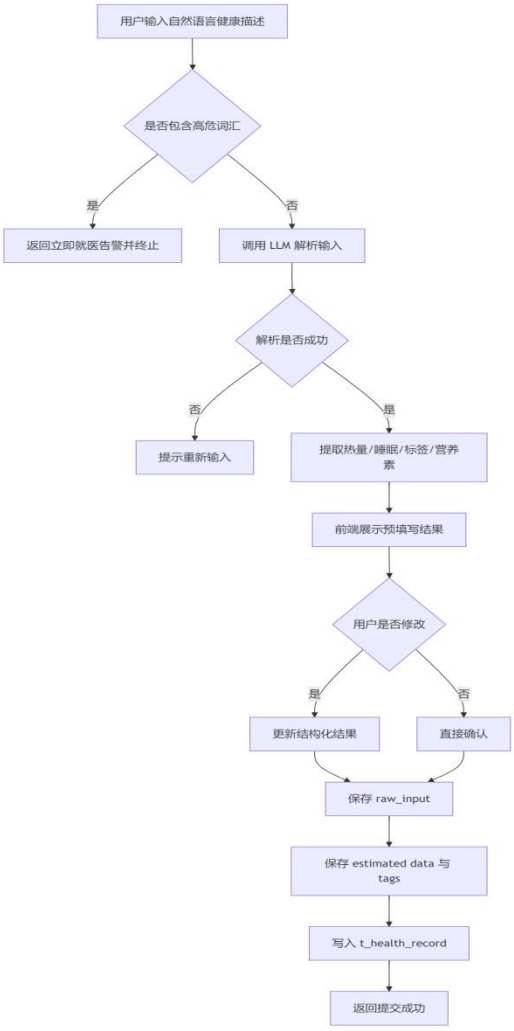
# 5 复杂功能算法设计

## 5.1 自然语言健康数据解析与双轨存储算法

### 5.1.1 设计目标

1. 模糊解析自然语言输入，转化为结构化数据；
2. 支持用户修正解析结果；
3. 双轨存储原始输入与结构化数据；
4. 高危词汇拦截，防控医疗风险。

### 5.1.2 算法流程图



### 5.1.3 伪代码实现

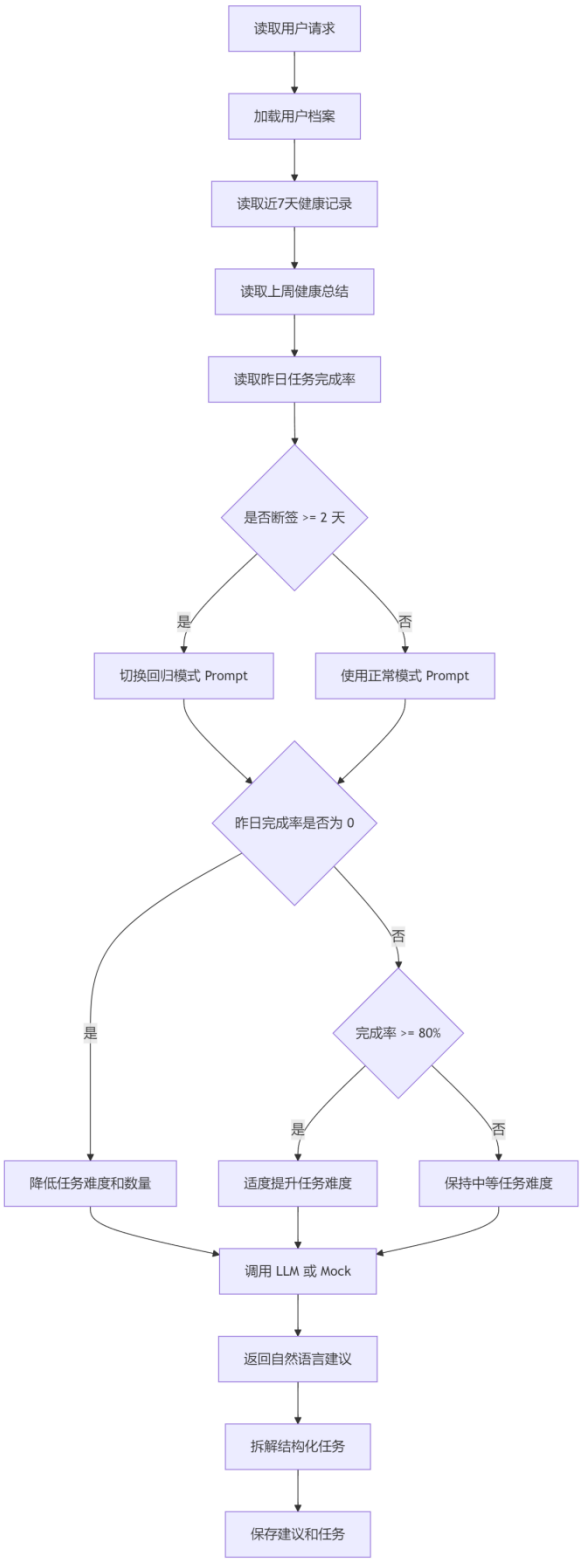
```Python
# 自然语言解析与风险校验
def parse_health_input(user_id: int, raw_input: str) -> dict:
    high_risk_words = ["胸痛", "晕厥", "吐血"]  # 高危词汇库
    if any(word in raw_input for word in high_risk_words):
        return {"code": -1, "message": "检测到高危症状，请立即就医", "data": None}

    llm_response = call_llm_api({"user_input": raw_input}, timeout=3)
    if not llm_response or llm_response.get("confidence") == "low":
        return {"code": -2, "message": "无法识别健康数据，请重新输入", "data": None}

    preview_data = {
        "raw_input": raw_input,
        "estimated_intake_kcal": llm_response.get("intake_kcal", 0),
        "estimated_burn_kcal": llm_response.get("burn_kcal", 0),
        "sleep_minutes": llm_response.get("sleep_minutes", 0),
        "nutrition_details": llm_response.get("nutrition_details", {}),
        "health_tags": llm_response.get("health_tags", []),
        "confidence": llm_response.get("confidence", "medium")
    }
    return {"code": 0, "message": "解析成功", "data": preview_data}

# 确认并保存双轨记录
def confirm_and_save_health_record(user_id: int, preview_data: dict, user_modified_data: dict
= None) -> dict:
    final_data = preview_data.copy()
    if user_modified_data:
        for key, value in user_modified_data.items():
            if key in final_data and value is not None:
                final_data[key] = value

    insert_data = {
        "user_id": user_id,
        "record_date": datetime.now().strftime("%Y-%m-%d"),
        "raw_input": final_data.get("raw_input"),
        "estimated_intake_kcal": final_data.get("estimated_intake_kcal"),
        "estimated_burn_kcal": final_data.get("estimated_burn_kcal"),
        "sleep_minutes": final_data.get("sleep_minutes"),
        "nutrition_details": json.dumps(final_data.get("nutrition_details", {})),
        "health_tags": json.dumps(final_data.get("health_tags", [])),
        "confidence": final_data.get("confidence")
    }

    try:
        with db_transaction():
            db.session.execute(insert(t_health_record).values(**insert_data))
            db.session.commit()
        return {"code": 0, "message": "提交成功", "data": {"record_id":
insert_data.get("record_id")}}
    except Exception as e:
        db.session.rollback()
        return {"code": -3, "message": f"数据保存失败：{str(e)}", "data": None}
```

## 5.2 AI 建议生成与任务难度动态调整算法

### 5.2.1 设计目标

1. 生成个性化、连续、安全的健康建议；
2. 动态调整任务难度与数量；
3. 支持断签回归模式；
4. 实现建议到任务的自动拆解；
5. 集成 Mock 兜底机制。

### 5.2.2 算法流程图


读取用户请求
加载用户档案
读取近7天健康记录
读取上周健康总结
读取昨日任务完成率
是否断签 >= 2 天
是
否
切换回归模式 Prompt
使用正常模式 Prompt
昨日完成率是否为 0
是
否
完成率 >= 80%
是
否
降低任务难度和数量
适度提升任务难度
保持中等任务难度
调用 LLM 或 Mock
返回自然语言建议
拆解结构化任务
保存建议和任务

### 5.2.3 伪代码实现

```
Python
# 判断断签≥2 天
def is_missing_record_ge2_days(user_id: int) -> bool:
    today = datetime.now().strftime("%Y-%m-%d")
    yesterday = (datetime.now() - timedelta(days=1)).strftime("%Y-%m-%d")
    record_count = db.session.query(t_health_record).filter(
        t_health_record.user_id == user_id,
        t_health_record.record_date.in_([today, yesterday])
    ).count()
    return record_count == 0

# 核心生成函数
def generate_daily_advice(user_id: int) -> dict:
    # 读取上下文数据
    user_profile = db.session.query(t_user).filter(t_user.user_id ==
user_id).first().to_dict()
    recent_records = db.session.query(t_health_record).filter(
        t_user.user_id == user_id,
        t_health_record.record_date >= (datetime.now() - timedelta(days=7)).strftime("%Y-
%m-%d")
    ).all()
    weekly_summary = redis_client.get(f"weekly_summary_{user_id}") or ""
    yesterday_tasks = db.session.query(t_daily_task).filter(
        t_daily_task.user_id == user_id,
        t_daily_task.task_date == (datetime.now() - timedelta(days=1)).strftime("%Y-%m-%d")
    ).all()

    # 计算完成率
    yesterday_completion_rate = 1.0 if len(yesterday_tasks) == 0 else sum(1 for t in
yesterday_tasks if t.status == 1) / len(yesterday_tasks)

    # 模式与难度设置
    if is_missing_record_ge2_days(user_id):
        mode_prompt = "回归模式：生成轻量恢复性建议，不追究未完成项"
    else:
        mode_prompt = "正常模式：结合近期数据与完成情况生成建议"

    if yesterday_completion_rate == 0.0:
        task_difficulty, task_count = "低", 1
        difficulty_prompt = f"1 个极轻量任务，增加情绪安慰"
    elif yesterday_completion_rate >= 0.8:
        task_difficulty, task_count = "高", 3
        difficulty_prompt = f"3 个有挑战性任务，提升健康要求"
    else:
        task_difficulty, task_count = "中", 2
        difficulty_prompt = f"2 个适中任务，保持当前要求"

    # 组装 Prompt
    system_constraint = "非医疗级建议，禁止诊断与处方药推荐，任务需含 task_content 与 ai_reason"
    context_prompt = f"用户档案：{json.dumps(user_profile)}；近 7 天记录：{[r.to_dict() for r
in recent_records]}；上周总结：{weekly_summary}；昨日完成率：
{yesterday_completion_rate*100}%"
    final_prompt =
f"{system_constraint}\n{context_prompt}\n{mode_prompt}\n{difficulty_prompt}\n 输出：200 字内
建议+JSON 任务数组"
```

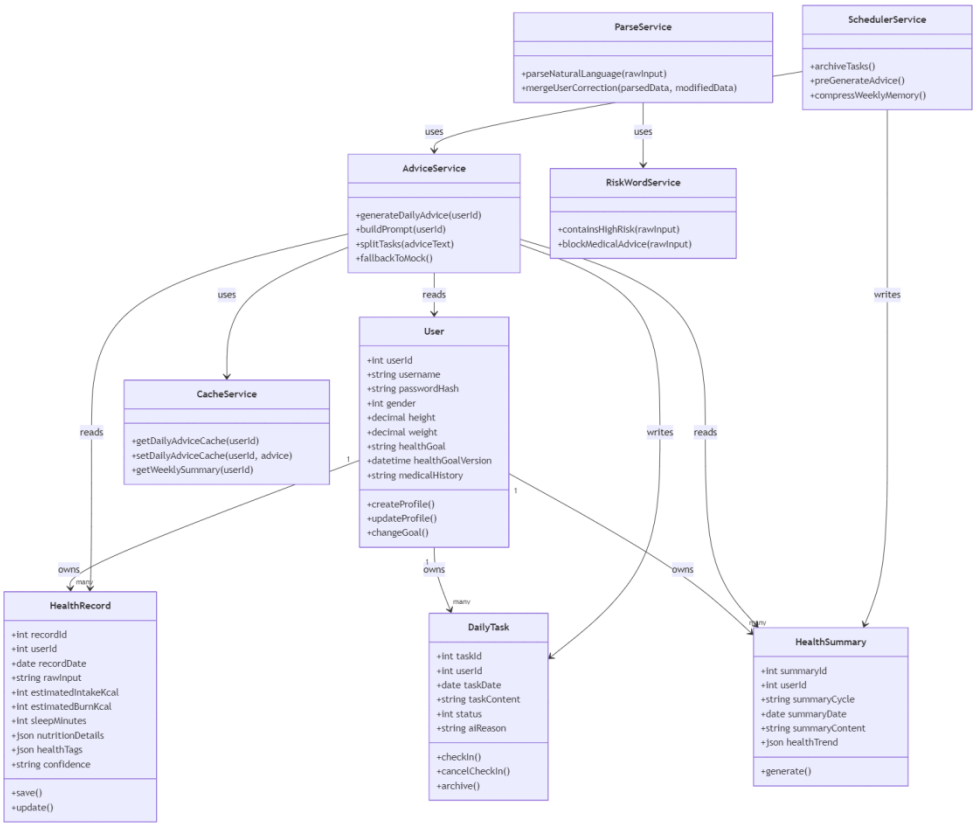
```
    # 调用 LLM/Mock
    USE_MOCK_AI = os.getenv("USE_MOCK_AI") == "True"
    if USE_MOCK_AI:
        advice_text = "今日建议规律作息、多喝水、适度运动"
        mock_tasks = [{"task_content": "23:30 前入睡", "ai_reason": "规律作息是健康基础
"}][:task_count]
        task_list = mock_tasks
    else:
        llm_response = call_llm_api(prompt=final_prompt, timeout=5,
response_format="text+json")
        if not llm_response:
            advice_text = "通用建议：饮食均衡、适度运动、充足睡眠"
            task_list = [{"task_content": "保证 7 小时睡眠", "ai_reason": "充足睡眠至关重要"}]
        else:
            advice_text = llm_response.split("\n")[0]
            task_list = json.loads(llm_response.split("\n")[1])

    # 保存与缓存
    try:
        with db_transaction():
            for task in task_list:
                db.session.execute(insert(t_daily_task).values(
                    user_id=user_id,
                    task_date=datetime.now().strftime("%Y-%m-%d"),
                    task_content=task.get("task_content"),
                    status=0,
                    ai_reason=task.get("ai_reason")
                ))
            db.session.commit()
    except Exception as e:
        db.session.rollback()
        return {"code": -1, "message": f"保存失败：{str(e)}", "data": None}

    redis_client.set(f"daily_advice_{user_id}_{datetime.now().strftime('%Y-%m-%d')}",
                     json.dumps({"advice_text": advice_text, "task_list": task_list}),
ex=86400)

    return {"code": 0, "message": "生成成功", "data": {"advice_text": advice_text,
"task_list": task_list}}
```

# 6 面向对象类设计

## 6.1 类设计核心思路

1. 实体类与服务类分离：实体类封装数据，服务类实现业务逻辑；
2. 服务类模块化：按功能划分，职责单一；
3. 依赖注入与接口化：降低耦合，支持扩展；
4. 贴合数据库表结构：通过 ORM 映射，简化开发。

## 6.2 核心类图设计




## 6.3 主要类职责详细说明

### 6.3.1 实体类

### User（用户类）

- 核心职责：封装用户身份信息与健康档案，提供档案创建、更新、目标修改方法。

### HealthRecord（健康记录类）

- 核心职责：封装健康数据，实现双轨存储，提供保存、更新方法。

### DailyTask（每日任务类）

- 核心职责：封装打卡任务，提供打卡、取消打卡、归档方法。

### HealthSummary（健康总结类）

- 核心职责：封装健康总结数据，提供生成、保存方法。

## 6.3.2 服务类

### AdviceService（建议服务类）

- 核心职责：生成 AI 建议、组装 Prompt、拆解任务、Mock 兜底。

### ParseService（解析服务类）

- 核心职责：解析自然语言输入、合并用户修改数据。

### RiskWordService（风险词服务类）

- 核心职责：检测高危词汇、拦截医疗相关输入、更新词汇库。

### CacheService（缓存服务类）

- 核心职责：管理 Redis 缓存，提供增删查操作。

### SchedulerService（调度服务类）

- 核心职责：调度定时任务，实现周总结压缩、建议预生成、任务归档。

# 7 接口设计

## 7.1 接口设计原则

1. 遵循 RESTful 风格，HTTP 方法对应操作类型，地址命名规范；
2. 统一 JSON 数据交换格式；
3. 强制 JWT 鉴权（除注册/登录）；
4. 统一响应格式，包含状态码、提示信息、业务数据；
5. 前后端双重参数校验；
6. 中文错误提示；
7. 关键接口实现幂等性。

## 7.2 统一响应格式

### 7.2.1 成功响应

```JSON
{
  "code": 0,
  "message": "操作成功",
  "data": {
    "key1": "value1",
    "key2": "value2"
  }
}
```

### 7.2.2 失败响应

```JSON
{
  "code": -1,
  "message": "用户名已被占用",
  "data": null
}
```

## 7.3 核心接口清单

### 7.3.1 用户管理接口

| 接口地址 | 请求方法 | 是否鉴权 | 请求参数 | 响应数据 | 接口说明 |
| --- | --- | --- | --- | --- | --- |
| /api/user/register | POST | 否 | {"username": "string", "password": "string", "confirmPassword": "string"} | {"code":0,"message":"注册成功","data":{"userId":1,"username":"xxx"}} | 用户注册，密码加密存储 |
| /api/user/login | POST | 否 | {"username": "string", "password": "string"} | {"code":0,"message":"登录成功","data":{"token":"xxx","userId":1,"username":"xxx"}} | 生成 JWT Token |
| /api/user/profile/create | POST | 是 | {"gender": 0/1, "height": 175.0, "weight": 65.0, "healthGoal": "减脂", "medicalHistory": "无"} | {"code":0,"message":"档案创建成功","data":null} | 新用户强制完成 |
| /api/user/profile/update | PUT | 是 | {"height": 175.0, "weight": 64.0, "medicalHistory": "无"} | {"code":0,"message":"档案更新成功","data":null} | 支持部分字段修改 |
| /api/user/goal/change | PUT | 是 | {"newGoal": "增肌"} | {"code":0,"message":"目标修改成功","data":null} | 同步更新目标版本 |

### 7.3.2 健康记录接口

| 接口地址 | 请求方法 | 是否鉴权 | 请求参数 | 响应数据 | 接口说明 |
|---|---|---|---|---|---|
| /api/health/record/natural | POST | 是 | {"rawInput": "跑了 40 分钟，吃了一个苹果"} | {"code":0,"message":"解析成功","data":{"previewData":{...}}} | 自然语言录入解析 |
| /api/health/record/confirm | POST | 是 | {"previewData": {...}, "userModifiedData": {...}} | {"code":0,"message":"提交成功","data":{"recordId":1}} | 确认并保存双轨数据 |
| /api/health/record/manual | POST | 是 | {"intakeKcal": 500, "burnKcal": 300, "sleepMinutes": 420, "healthTags": ["有氧"]} | {"code":0,"message":"提交成功","data":{"recordId":1}} | 手动表单录入 |
| /api/health/record/recent | GET | 是 | query：days=7 | {"code":0,"message":"查询成功","data":{"records":[...]}} | 查询近期记录 |

### 7.3.3 AI 建议与任务接口

| 接口地址 | 请求方法 | 是否鉴权 | 请求参数 | 响应数据 | 接口说明 |
|---|---|---|---|---|---|
| /api/health/daily-report | GET | 是 | 无 | SSE 流式响应 | 获取今日 AI 建议 |
| /api/task/check | POST | 是 | {"taskId": 1, "status": 1/0} | {"code":0,"message":"打卡成功","data":null} | 任务打卡/取消 |
| /api/task/today | GET | 是 | 无 | {"code":0,"message":"查询成功","data":{"tasks":[...]}} | 查询今日任务 |
| /api/task/history | GET | 是 | query：date=2026-03-24 | {"code":0,"message":"查询成功","data":{"tasks":[...]}} | 查询历史打卡记录 |

### 7.3.4 数据可视化接口

| 接口地址 | 请求方法 | 是否鉴权 | 请求参数 | 响应数据 | 接口说明 |
|---|---|---|---|---|---|
| /api/visual/dashboard | GET | 是 | 无 | {"code":0,"message":"查询成功","data":{"completionRate":0.8,...}} | 获取健康仪表盘数据 |
| /api/visual/trend | GET | 是 | query: type=week/month, indicator=sleep/kcal | {"code":0,"message":"查询成功","data":{"trendData":[...]}} | 获取周/月趋势数据 |
| /api/visual/tags | GET | 是 | query: days=7 | {"code":0,"message":"查询成功","data":{"tags":{"有氧":3}}} | 获取健康标签统计 |

## 7.4 接口安全设计

### 7.4.1 JWT 鉴权防护

- Token 有效期 2 小时，密钥后端存储，修改密码/登出时失效 Token。

### 7.4.2 输入参数校验

- 前端即时校验，后端 Pydantic 严格校验，自然语言输入额外拦截高危词汇。

### 7.4.3 接口幂等性

- 健康记录：用户 ID+日期唯一索引，重复提交更新原有记录；

- 任务打卡：校验当前状态，避免重复操作；
- 核心接口添加 requestId 缓存。

### 7.4.4 数据脱敏与访问控制

- 隐藏敏感信息，校验用户 ID 与资源归属一致性，接口限流（每分钟 10 次）。

### 7.4.5 异常处理与日志记录

- 全局异常处理，返回统一错误提示；
- 记录请求参数、响应结果、异常详情，便于排查。

# 8 数据库物理设计

## 8.1 数据库选型及理由

### 8.1.1 主数据库：MySQL 8.0

理由：开源稳定，支持事务、JSON 字段，与 SQLAlchemy 兼容，适配结构化数据存储。

### 8.1.2 缓存数据库：Redis 6.0

理由：高性能，支持多种数据结构与过期时间，适配高频数据缓存。

## 8.2 数据库表结构设计

### 8.2.1 用户表（t_user）

| 字段名 | 字段类型 | 长度 | 是否主键 | 是否非空 | 默认值 | 备注 |
|---|---|---|---|---|---|---|
| user_id | INT | 11 | 是 | 是 | AUTO_INCREMENT | 用户唯一标识 |
| username | VARCHAR | 50 | 否 | 是 | 无 | 登录用户名，唯一 |
| password_hash | VARCHAR | 255 | 否 | 是 | 无 | 密码 bcrypt 哈希 |
| gender | TINYINT | 1 | 否 | 是 | 无 | 0：女，1：男 |
| height | DECIMAL | 5,2 | 否 | 是 | 无 | 身高（cm） |
| weight | DECIMAL | 5,2 | 否 | 是 | 无 | 体重（kg） |
| health_goal | VARCHAR | 20 | 否 | 是 | 无 | 健康目标 |
| health_goal_version | DATETIME | 无 | 否 | 是 | CURRENT_TIMESTAMP | 目标最后修改时间 |
| medical_history | TEXT | 无 | 否 | 否 | 无 | 伤病史/过敏史 |
| created_at | DATETIME | 无 | 否 | 是 | CURRENT_TIMESTAMP | 创建时间 |
| updated_at | DATETIME | 无 | 否 | 是 | CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP | 更新时间 |

索引：idx_username（username 唯一索引）。

### 8.2.2 健康记录表（t_health_record）

| 字段名 | 字段类型 | 长度 | 是否主键 | 是否非空 | 默认值 | 备注 |
|---|---|---|---|---|---|---|
| record_id | INT | 11 | 是 | 是 | AUTO_INCREMENT | 记录唯一标识 |
| user_id | INT | 11 | 否 | 是 | 无 | 关联用户 ID，外键 |
| record_date | DATE | 无 | 否 | 是 | CURRENT_DATE | 记录日期 |
| raw_input | TEXT | 无 | 否 | 是 | 无 | 原始输入 |
| estimated_intake_kcal | INT | 11 | 否 | 是 | 0 | 摄入热量（大卡） |
| estimated_burn_kcal | INT | 11 | 否 | 是 | 0 | 消耗热量（大卡） |
| sleep_minutes | INT | 11 | 否 | 是 | 0 | 睡眠时长（分钟） |
| nutrition_details | JSON | 无 | 否 | 否 | NULL | 营养素详情 |
| health_tags | JSON | 无 | 否 | 否 | NULL | 健康标签 |
| confidence | VARCHAR | 20 | 否 | 是 | "medium" | 解析置信度 |
| updated_at | DATETIME | 无 | 否 | 是 | CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP | 更新时间 |

索引：idx_user_date（user_id+record_date）、idx_record_date（record_date）。

### 8.2.3 每日任务表（t_daily_task）

| 字段名 | 字段类型 | 长度 | 是否主键 | 是否非空 | 默认值 | 备注 |
|---|---|---|---|---|---|---|
| task_id | INT | 11 | 是 | 是 | AUTO_INCREMENT | 任务唯一标识 |
| user_id | INT | 11 | 否 | 是 | 无 | 关联用户ID，外键 |
| task_date | DATE | 无 | 否 | 是 | CURRENT_DATE | 任务日期 |
| task_content | VARCHAR | 255 | 否 | 是 | 无 | 任务内容 |
| status | TINYINT | 1 | 否 | 是 | 0 | 0：未完成，1：已完成，2：已归档 |
| ai_reason | VARCHAR | 255 | 否 | 是 | 无 | AI 生成缘由 |
| created_at | DATETIME | 无 | 否 | 是 | CURRENT_TIMESTAMP | 创建时间 |
| updated_at | DATETIME | 无 | 否 | 是 | CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP | 更新时间 |

索引：idx_user_task_date（user_id+task_date+status）、idx_task_date（task_date）。

### 8.2.4 健康总结表（t_health_summary）

| 字段名 | 字段类型 | 长度 | 是否主键 | 是否非空 | 默认值 | 备注 |
|---|---|---|---|---|---|---|
| summary_id | INT | 11 | 是 | 是 | AUTO_INCREMENT | 总结唯一标识 |
| user_id | INT | 11 | 否 | 是 | 无 | 关联用户ID，外键 |
| summary_cycle | VARCHAR | 10 | 否 | 是 | 无 | 总结周期：week/month |
| summary_date | DATE | 无 | 否 | 是 | 无 | 周期结束日期 |
| summary_content | TEXT | 无 | 否 | 是 | 无 | 总结内容（200字内） |
| health_trend | JSON | 无 | 否 | 否 | NULL | 健康趋势标签 |
| created_at | DATETIME | 无 | 否 | 是 | CURRENT_TIMESTAMP | 生成时间 |

索引：idx_user_cycle_date（user_id+summary_cycle+summary_date）。

## 8.3 主要索引设计

1. t_user：idx_username（唯一索引）；
2. t_health_record：idx_user_date（联合索引）、idx_record_date（普通索引）；
3. t_daily_task：idx_user_task_date（联合索引）、idx_task_date（普通索引）；

4. t_health_summary：idx_user_cycle_date（联合索引）。

## 8.4 表间关系设计

1. t_user 与 t_health_record：1:N，级联删除/更新；
2. t_user 与 t_daily_task：1:N，级联删除/更新；
3. t_user 与 t_health_summary：1:N，级联删除/更新。

## 8.5 物理建表 SQL 示例

```
SQL
-- 用户表
CREATE TABLE IF NOT EXISTS t_user (
    user_id INT(11) NOT NULL AUTO_INCREMENT COMMENT '用户唯一标识',
    username VARCHAR(50) NOT NULL COMMENT '登录用户名，唯一',
    password_hash VARCHAR(255) NOT NULL COMMENT '密码 bcrypt 加盐哈希',
    gender TINYINT(1) NOT NULL COMMENT '0：女，1：男',
    height DECIMAL(5,2) NOT NULL COMMENT '身高（cm）',
    weight DECIMAL(5,2) NOT NULL COMMENT '体重（kg）',
    health_goal VARCHAR(20) NOT NULL COMMENT '健康目标',
    health_goal_version DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '目标最后修改时
间',
    medical_history TEXT COMMENT '伤病史/过敏史',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
COMMENT '更新时间',
    PRIMARY KEY (user_id),
    UNIQUE KEY idx_username (username)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表';

-- 健康记录表
CREATE TABLE IF NOT EXISTS t_health_record (
    record_id INT(11) NOT NULL AUTO_INCREMENT COMMENT '记录唯一标识',
    user_id INT(11) NOT NULL COMMENT '关联用户 ID',
    record_date DATE NOT NULL DEFAULT CURRENT_DATE COMMENT '记录日期',
    raw_input TEXT NOT NULL COMMENT '原始输入',
    estimated_intake_kcal INT(11) NOT NULL DEFAULT 0 COMMENT '摄入热量',
    estimated_burn_kcal INT(11) NOT NULL DEFAULT 0 COMMENT '消耗热量',
    sleep_minutes INT(11) NOT NULL DEFAULT 0 COMMENT '睡眠时长（分钟）',
    nutrition_details JSON COMMENT '营养素详情',
    health_tags JSON COMMENT '健康标签',
    confidence VARCHAR(20) NOT NULL DEFAULT 'medium' COMMENT '解析置信度',
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
COMMENT '更新时间',
    PRIMARY KEY (record_id),
    KEY idx_user_date (user_id, record_date),
    KEY idx_record_date (record_date),
    CONSTRAINT fk_health_record_user FOREIGN KEY (user_id) REFERENCES t_user (user_id) ON
DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='健康记录表';

-- 每日任务表
CREATE TABLE IF NOT EXISTS t_daily_task (
    task_id INT(11) NOT NULL AUTO_INCREMENT COMMENT '任务唯一标识',
    user_id INT(11) NOT NULL COMMENT '关联用户 ID',
    task_date DATE NOT NULL DEFAULT CURRENT_DATE COMMENT '任务日期',
    task_content VARCHAR(255) NOT NULL COMMENT '任务内容',
    status TINYINT(1) NOT NULL DEFAULT 0 COMMENT '0 未完成，1 已完成，2 已归档',
    ai_reason VARCHAR(255) NOT NULL COMMENT 'AI 生成缘由',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
COMMENT '更新时间',
    PRIMARY KEY (task_id),
```

```
    KEY idx_user_task_date (user_id, task_date, status),
    KEY idx_task_date (task_date),
    CONSTRAINT fk_daily_task_user FOREIGN KEY (user_id) REFERENCES t_user (user_id) ON
DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='每日任务表';

-- 健康总结表
CREATE TABLE IF NOT EXISTS t_health_summary (
    summary_id INT(11) NOT NULL AUTO_INCREMENT COMMENT '总结唯一标识',
    user_id INT(11) NOT NULL COMMENT '关联用户 ID',
    summary_cycle VARCHAR(10) NOT NULL COMMENT '总结周期',
    summary_date DATE NOT NULL COMMENT '周期结束日期',
    summary_content TEXT NOT NULL COMMENT '总结内容',
    health_trend JSON COMMENT '健康趋势标签',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '生成时间',
    PRIMARY KEY (summary_id),
    KEY idx_user_cycle_date (user_id, summary_cycle, summary_date),
    CONSTRAINT fk_health_summary_user FOREIGN KEY (user_id) REFERENCES t_user (user_id) ON
DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='健康总结表';
```

# 9 UI 界面与交互设计

## 9.1 UI 设计核心原则

1. 现代简约，轻医美视觉；
2. 低饱和蓝绿色为主色调，突出健康、专业调性；
3. 卡片式布局，强化信息层级；
4. 核心操作路径最短化。

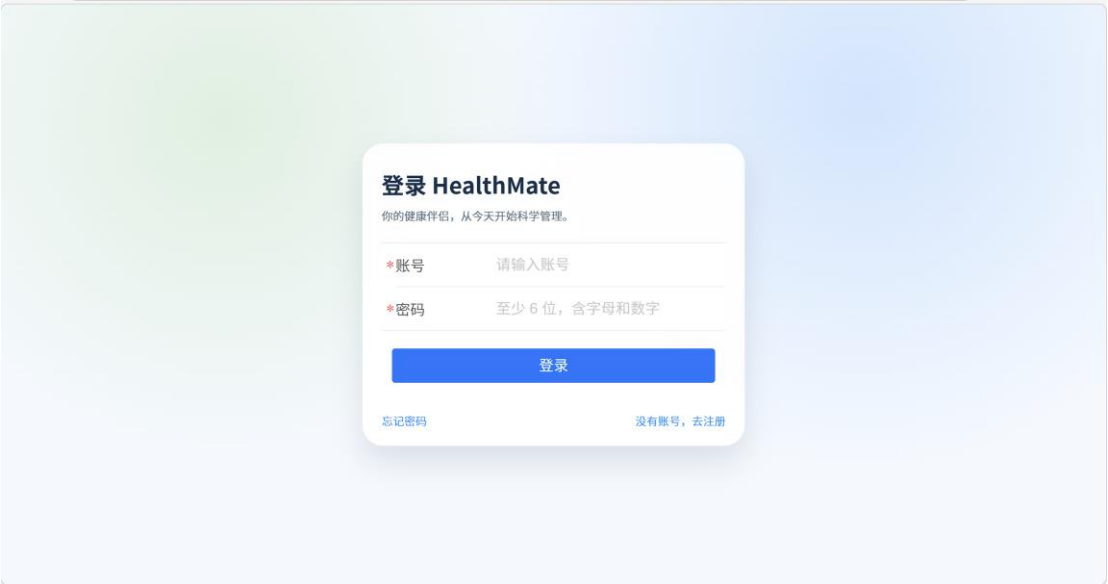
## 9.2 视觉规范

- 主色：#409EFF；辅助色：#67C23A；警告色：#F56C6C；
- 背景色：白色、浅灰色、浅蓝绿色渐变；
- 圆角：16px 左右；阴影：轻阴影。

## 9.3 核心页面结构设计

### 9.3.1 注册/登录页

- 居中布局，简洁表单，含账号、密码输入框，支持显示/隐藏密码；
- 注册页含密码强度提示，页底提供切换入口。



## 9.3.2 基础健康档案填写页

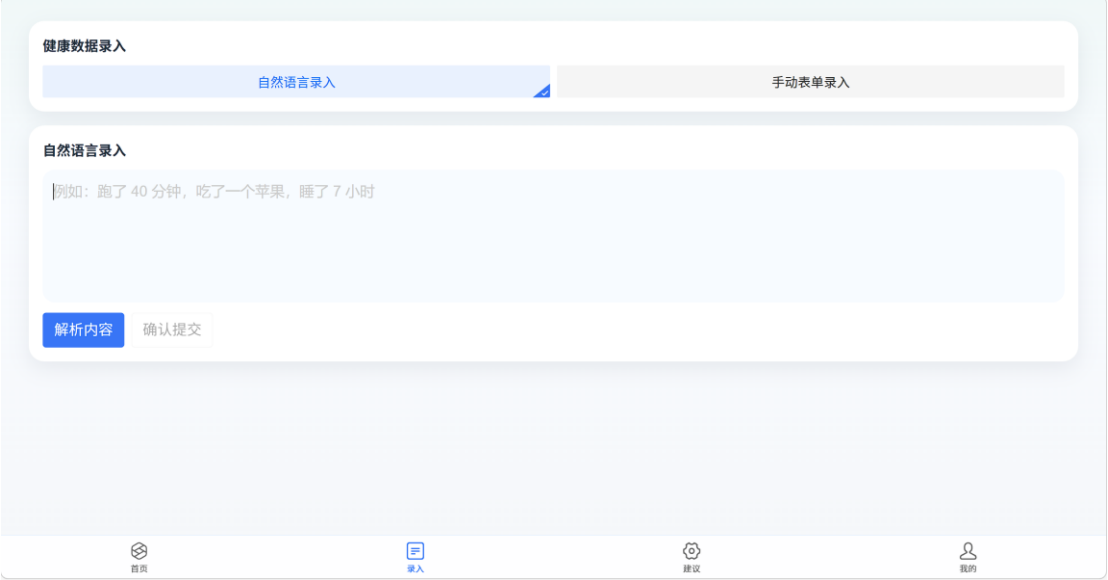
- 单栏表单，必填项：性别、身高、体重、健康目标；
- 伤病史/过敏史用多行文本框。

## 9.3.3 首页健康仪表盘

- 顶部：用户信息与当前目标；
- 核心区域：今日任务完成率环形图、近 7 天睡眠/热量趋势图；
- 底部固定导航：首页、录入、建议、个人中心。

## 9.3.4 健康数据录入页

- 标签切换自然语言/手动录入；
- 自然语言录入用大文本框，解析结果卡片式展示，支持修正；
- 高危词汇输入弹窗告警，禁用提交。

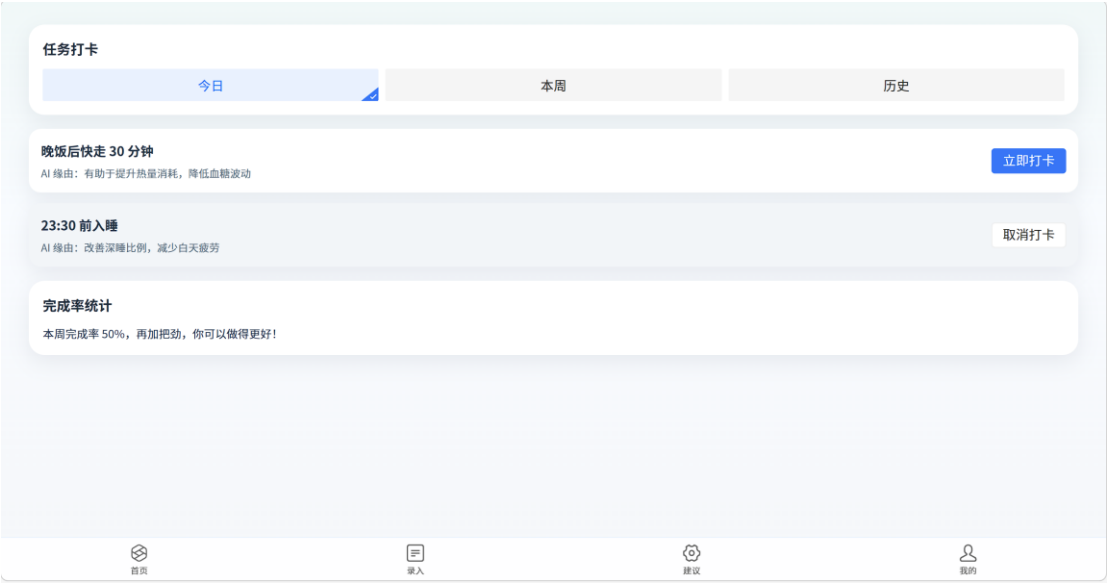



## 9.3.5 AI 健康建议页

- 主区域：SSE 流式输出建议文本；
- 下方：1-3 个任务卡片，含 AI 缘由与打卡按钮；
- 页底：返回首页与历史建议入口。

## 9.3.6 任务打卡页

- 顶部支持“今日/本周/历史”筛选；
- 卡片区分待办/已完成任务，底部显示完成率与评价。

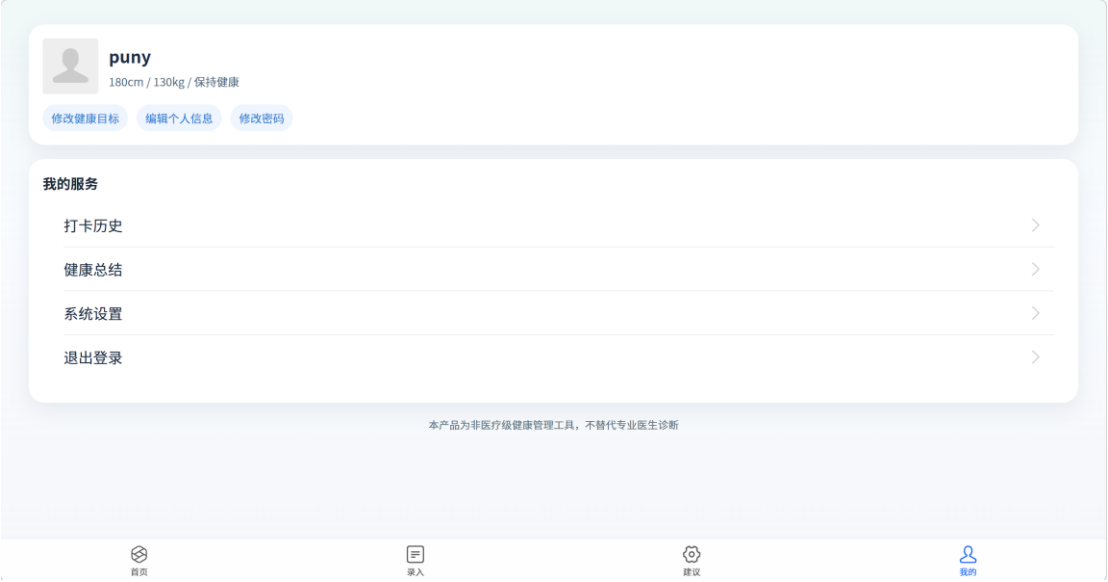



## 9.3.7 健康数据可视化趋势页

- 顶部：周/月维度切换与日期筛选；
- 左侧：趋势折线图；右侧：标签饼图/词云；
- 提供数据导出入口。

### 9.3.8 个人中心页

- 顶部：用户画像与当前目标；
- 中部：修改目标、编辑信息等入口；
- 底部：免责声明。




## 9.4 关键交互设计

1. 表单即时校验，中文错误提示；
2. AI 建议流式输出显示加载状态，网络中断支持断点续传；
3. 打卡按钮点击后即时动效反馈；
4. 折线图悬浮查看数值，异常点标红；

5. 高危词汇输入立即阻断流程。